



Red Hat OpenShift AI Self-Managed 3.4

Working with MLflow

Work with MLflow from Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.4 Working with MLflow

Work with MLflow from Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As a data scientist or platform engineer, you can use the MLflow operator in OpenShift AI.

Table of Contents

CHAPTER 1. ABOUT MLFLOW	3
CHAPTER 2. INSTALL AND CONFIGURE MLFLOW	4
2.1. MLFLOW AND MLFLOWCONFIG CONFIGURATION PARAMETERS	6
2.2. AGGREGATE CLUSTER ROLES	7
2.3. RBAC MODEL FOR MLFLOW API USAGE	8
CHAPTER 3. INSTALL AND AUTHENTICATE THE MLFLOW SDK	9
3.1. CONFIGURING THE MLFLOW SDK FOR A LOCAL WORKSTATION	10
3.2. CONFIGURING MLFLOW SDK ENVIRONMENT VARIABLES FOR PODS	10
3.3. UPSTREAM MLFLOW SDK REFERENCE	11
3.4. MLFLOW SDK TROUBLESHOOTING REFERENCE	11
3.5. MLFLOW VERSION COMPATIBILITY	13
3.6. MLFLOW STORAGE AND DATABASE COMPATIBILITY	13
3.7. TRACKING EXPERIMENTS WITH MLFLOW SDK	13
3.8. CONFIGURE PROJECT-SPECIFIC S3 ARTIFACT STORAGE	14

CHAPTER 1. ABOUT MLFLOW

Red Hat OpenShift AI deploys a single shared MLflow instance through the MLflow operator. With this deployment, you can use MLflow workspaces and Kubernetes-backed authorization. For more information, see [MLflow Workspaces](#).

A project is a namespace that has its own set of objects, policies, constraints, and service accounts. Each Red Hat OpenShift Container Platform project maps to an MLflow workspace in a one-to-one relationship. MLflow provides logical isolation of experiments, runs, registered models, prompts, datasets, and traces per workspace.

OpenShift AI manages the workspace lifecycle outside of MLflow. The MLflow API does not create, update, or delete workspaces. Instead, you manage workspaces through the OpenShift AI dashboard or the Red Hat OpenShift command-line interface (CLI). When you manage projects through these tools, the corresponding MLflow workspace becomes available automatically.

Every MLflow API request relies on Kubernetes role-based access control (RBAC) for authorization. The MLflow server submits a SelfSubjectAccessReview against pseudo-resources in the `mlflow.kubeflow.org` API group, using the caller's bearer token and the target project namespace. Users who already have the Red Hat OpenShift admin, edit, or view roles on a project automatically receive the corresponding MLflow permissions.

CHAPTER 2. INSTALL AND CONFIGURE MLFLOW

To track machine learning experiments in OpenShift AI, use the Red Hat OpenShift AI Operator to deploy a cluster-scoped **mlflow** instance.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift Container Platform
- You have permission to patch the **DataScienceCluster** resource and apply **MLflow** custom resources.
- For production-oriented deployments, you have created the required secrets for database credentials and S3-compatible storage.

Procedure

The MLflow resource is cluster scoped. Therefore, you can install only one instance of **mlflow** in the cluster. The Red Hat OpenShift AI Operator creates the MLflow resource in the `redhat-ods-applications` namespace.

1. Log in to the OpenShift cluster by using the CLI.
2. Enable the MLflow Operator component by patching the **DataScienceCluster** object:

```
$ oc patch datasciencecluster default-dsc \
  --type=merge \
  -p {"spec":{"components":{"mlflowoperator":{"managementState":"Managed"}}}}
```

3. Create an **MLflow** custom resource (CR). Choose the configuration that matches your environment:

Minimal development or test deployment

Uses SQLite for the backend store and a persistent volume claim (PVC) for file-based artifact storage.

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
  name: mlflow
  namespace: redhat-ods-applications
spec:
  storage:
    accessModes:
      - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  backendStoreUri: "sqlite:///mlflow/mlflow.db"
  artifactsDestination: "file:///mlflow/artifacts"
  serveArtifacts: true
```

Production-oriented deployment

Uses PostgreSQL for metadata and S3-compatible object storage for artifacts.

```
apiVersion: mlflow.opendatahub.io/v1
kind: MLflow
metadata:
  name: mlflow
  namespace: redhat-ods-applications
spec:
  replicas: 2
  backendStoreUriFrom:
    name: mlflow-db-credentials
    key: backend-store-uri
  artifactsDestination: "s3://my-mlflow-bucket/artifacts"
  serveArtifacts: true
  envFrom:
    - secretRef:
        name: aws_access_key_id=<your_aws_access_key>
          aws_secret_access_key=<your_aws_secret_access_key>
          aws_session_token=<your_aws_session_token>
```

Table 2.1. MLflow Deployment and Artifact Configuration

Parameter or Feature	Configuration Details
replicas	Acts as an important scaling knob specifically for production-oriented deployments.
registryStoreUri	Defaults to the value of backendStoreUri if not specified. In practice, the same PostgreSQL database is typically utilized for both.
serveArtifacts	When enabled, clients log and retrieve artifacts through the MLflow server REST API via the mlflow-artifacts:/ proxy URI scheme. If defaultArtifactRoot is not specified while this is enabled, it automatically defaults to mlflow-artifacts:/ .
defaultArtifactRoot	Defines the directory for storing artifacts for each new experiment. Do not set this to a direct-storage URI (e.g., s3://) when serveArtifacts is enabled; doing so causes clients to bypass the proxy and attempt direct storage access.
Artifact Storage Path	If no per-project override is configured, artifacts are stored under <default_artifact_root>/workspaces/<project_name> . Note that file-based artifact storage requires serveArtifacts: true .

4. In the OpenShift AI dashboard navigation bar, click the **Applications** menu.
5. Confirm that the **MLflow UI** is displayed in the list.
6. Open the UI, select your workspace, and verify that you can compare runs and visualize metrics.
7. Alternatively, click **Start Demo**, which loads demo data to compare runs and visualize metrics.

Additional resources

- To view all available configuration options for the MLflow custom resource, run the **oc explain mlflow.spec** command.

2.1. MLFLOW AND MLFLOWCONFIG CONFIGURATION PARAMETERS

The MLflow deployment on Red Hat OpenShift AI relies on two custom resources: **MLflow** and **MLflowConfig**. These resources configure metadata storage, artifact destinations, and project-specific overrides.

The **MLflow** resource is cluster-scoped and must be named **mlflow**. This resource defines the global state of the MLflow deployment, including the database backend, artifact storage, replica count, and TLS configuration.

Table 2.2. Key fields in the MLflow resource

Field	Description
spec.replicas	The number of MLflow pods. The default value is 1. High-availability deployments require remote storage because PVC-backed storage does not support multiple concurrent writers.
spec.backendStoreUri	The tracking metadata database URI. Supported values include sqlite:// for development and postgresql:// for production. For production usage, use spec.backendStoreUriFrom to avoid storing database credentials in the MLflow CR.
spec.backendStoreUriFrom	A secret reference for the backend database URI. This field is used when the URI contains credentials.
mlflow.spec.resources	Specifies the compute resources for the MLflow container.

The **MLflowConfig** resource is namespace-scoped and must be named **mlflow**. Project owners can use this resource to override default artifact storage settings for their specific projects.

Table 2.3. Key fields in the MLflowConfig resource

Field	Description
spec.artifactRootSecret	The name of a secret in the project that contains S3-compatible connection credentials and bucket information. The value must be mlflow-artifact-connection .
spec.artifactRootPath	An optional relative path appended to the bucket root from the secret. For example, if the bucket is my-bucket and the path is mlflow-artifacts , the resolved artifact root is s3://my-bucket/mlflow-artifacts .

- To view all available configuration fields for a resource, use the **oc explain <resource>.spec** command.

2.2. AGGREGATE CLUSTER ROLES

MLflow uses aggregate cluster roles to control access to MLflow resources in Red Hat OpenShift AI. These roles determine which MLflow operations a user can perform based on their existing role bindings.



IMPORTANT

Standard bindings for **view**, **edit**, and **admin** roles do not grant identical access to all MLflow resources. While **mlflow** resources are cluster-scoped, **mlflowconfig** resources are namespace-scoped. Effective access depends on whether permissions are granted at the cluster level or the namespace level.

The **mlflow-view** ClusterRole aggregates into the standard OpenShift AI **view**, **edit**, and **admin** roles. User access depends on the scope of the binding set at the cluster level.

This role grants read-only access to the following MLflow resources:

- **get**, **list**, and **watch** permissions for the **mlflow** and **mlflowconfigs** Custom Resource Definitions (CRDs)
- **get** and **list** permissions for MLflow pseudo-resources, such as **datasets**, **experiments**, and **registeredmodels**
- **get** permissions for MLflow status

The **mlflow-edit** ClusterRole aggregates into the standard OpenShift AI **edit** and **admin** roles. User access depends on the scope of the binding set at the cluster level.

This role grants write access to the following MLflow resources:

- **create**, **read**, **update**, and **delete** permissions for the **mlflow** CRD
- **create**, **delete**, **patch**, and **update** permissions for the **mlflowconfigs** CRD
- **create**, **update**, and **delete** permissions for MLflow pseudo-resources
- **update** permissions for **finalizers**

The **mlflow-integration** ClusterRole grants service accounts data-plane access to MLflow without requiring full edit or delete privileges. When you bind this role with a RoleBinding, it provides namespace-scoped access. This ClusterRole does not grant **delete** or **access** to the control-plane resources.

This role is intended for integration-focused tasks and grants the following access:

- **get**, **list**, **create**, and **update** permissions for the **experiments**, **datasets**, and **registeredmodels** authorization-plugin pseudo-resources
- Read access to MLflow resources and statuses
- Permissions to interact with the MLflow tracking API and model registry

2.3. RBAC MODEL FOR MLFLOW API USAGE

The MLflow server authorizes every API request by using a Kubernetes **SelfSubjectAccessReview**. The server uses the caller's bearer token to determine if the token has permission to perform a specific verb on an MLflow pseudo-resource within the target project namespace.

The authorization checks use the **mlflow.kubeflow.org** API group. These *pseudo-resources* are used solely for role-based access control (RBAC) policy evaluation. Pseudo-resources are not actual Kubernetes resources or Custom Resource Definitions (CRDs), they do not exist as objects on the cluster. You cannot create, list, or inspect pseudo-resources by using the Kubernetes API or command-line tools such as **kubect**l.

Access is granted through standard Kubernetes **Role**, **ClusterRole**, **RoleBinding** and **ClusterRoleBinding** objects.

The workspace name serves as the Kubernetes namespace for access checks. The MLflow API access is project-scoped.

The following table outlines primary pseudo-resources:

Table 2.4. MLflow pseudo-resources

Pseudo-resource	Controls access to
experiments	Experiments, runs, traces, artifacts, logged models, scorers, and related tracking operations.
registeredmodels	Registered models, model versions, and prompts.
datasets	Evaluation datasets and related data set operations.

+ .Role assignments for MLflow API usage

The Red Hat OpenShift AI admin, edit, and view roles include the permissions required for MLflow API authorization checks through aggregate **ClusterRoles**. These roles allow users to interact with MLflow components within their assigned namespaces.

Workloads or agents that require MLflow API access without using broad human-facing roles can use built-in integration roles. For example, a dedicated mlflow-integration **ClusterRole** is available for service accounts that need to perform automated tracking or model registry tasks.

+ .Resource-name granularity The MLflow Kubernetes authorization plugin allows you to assign permissions with **resourceName** granularity. For example, you can restrict an agent so that it sends traces to only a specific experiment by defining the following attributes in the RBAC policy:

- **resourceName**: The name of the specific experiment
- **resource**: **experiments**
- **apiGroup**: **mlflow.kubeflow.org**
- **verbs**: **get, list, update**

CHAPTER 3. INSTALL AND AUTHENTICATE THE MLFLOW SDK

Install the MLflow SDK and configure authentication for your OpenShift cluster to track machine learning experiments in Red Hat OpenShift AI.

Prerequisites

- You have access to a OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) and are able to access MLflow in your OpenShift AI project or MLflow workspace.
- For the automated authentication method, you have a service account with the appropriate permissions or are running locally.

Procedure

1. Install the MLflow SDK:

```
pip install "mlflow[kubernetes]>=3.11"
```

1. Authenticate the SDK by using one of the following methods:

- a. To use automated authentication, enable the **kubernetes-namespaced** client-side authentication plugin.



NOTE

This plugin reads credentials from the mounted service account token when running in a pod, or from the active **kubeconfig** context when running on a workstation. The MLflow workspace is automatically configured by reading either the service account's namespace or active **kubeconfig** namespace.

Enter the following commands:

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_AUTH=kubernetes-namespaced
```

- b. To authenticate manually, export your tracking token and project workspace as environment variables. Enter the following commands:

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_TOKEN="$(oc whoami --show-token)"
export MLFLOW_WORKSPACE="<project-name>"
```



IMPORTANT

This configuration example is not recommended for production environments.

2. If your OpenShift cluster does not use trusted TLS certificates, enter the following command to disable TLS verification:

```
export MLFLOW_TRACKING_INSECURE_TLS=true
```

Verification

Use the following command to verify connectivity **python -c "import mlflow; print(mlflow.list_workspaces())"** This command lists the workspaces you can access on the OpenShift cluster.

3.1. CONFIGURING THE MLFLOW SDK FOR A LOCAL WORKSTATION

When you run the MLflow SDK on a local workstation, the authentication plugin uses the active kubeconfig context. The plugin uses the namespace from the kubeconfig context as the workspace and resolves the authentication token from your kubeconfig credentials, including exec-based authentication providers used by **oc login** command.

+ .Prerequisites

- You have installed the MLflow SDK.
- You have an active kubeconfig context.

Procedure

- + . Set the following environment variables to configure the tracking URI and authentication method:

+

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_AUTH=kubernetes-namespaced
```

1. Optional: If you prefer to set the token and workspace manually, export the following variables:

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_TOKEN="$(oc whoami --show-token)"
export MLFLOW_WORKSPACE="<project-name>"
```

3.2. CONFIGURING MLFLOW SDK ENVIRONMENT VARIABLES FOR PODS

- + When you run a pod, the Kubernetes authentication plugin uses the mounted service account token and namespace to automatically set the workspace.

You do not have to call **mlflow.set_workspace()** when you enable the authentication plugin. The plugin derives the workspace from the pod's service account namespace. You can override the workspace explicitly if you need to target a different project, provided that your service account has the necessary RBAC permissions in that project.

Prerequisites

- You have installed the MLflow SDK.

Procedure

1. Set the following environment variables in your pod configuration:

```
export MLFLOW_TRACKING_URI="https://<dashboard-url>/mlflow"
export MLFLOW_TRACKING_AUTH=kubernetes-namespaced
```

Verification

1. Run a Python script using the following code to confirm that the MLflow SDK successfully connects to the tracking server and logs data:

```
import mlflow

mlflow.set_experiment("demo-experiment")

with mlflow.start_run():
    mlflow.log_param("framework", "pytorch")
    mlflow.log_metric("accuracy", 0.95)
```

3.3. UPSTREAM MLFLOW SDK REFERENCE

Additional resources

For more information about the upstream MLflow SDK, see the following resources:

- [MLflow tracking](#)
- [MLflow Tracking API reference](#)
- [MLflow Model Registry](#)
- [Getting started with MLflow workspaces](#)
- [MLflow workspace providers](#)
- [MLflow 3 migration guide](#)
- [MLflow Python API reference](#)

3.4. MLFLOW SDK TROUBLESHOOTING REFERENCE

If you encounter errors when working with experiments or artifacts in the MLflow SDK, use the following information to resolve common issues and error messages.

If your issue is not described here, contact Red Hat support.

Common issues

403 or permission denied

Problem

The active project is missing the required role-based access control (RBAC) permissions.

Resolution

Verify that your user or service account has the necessary role binding in the active project.

Workspace not found

Problem

The SDK cannot locate the workspace because the project name is incorrect, the namespace is filtered, or no workspace was selected.

Resolution

Verify that your project name is correct and that the namespace is not restricted. Ensure that you have selected a workspace in your MLflow environment settings by using one of the following methods: * The **MLFLOW_TRACKING_AUTH=kubernetes-namespaced** environment variable. * The **MLFLOW_WORKSPACE=<workspace_name>** environment variable. * The **mlflow.set_workspace("team-a")** Python function.

Artifact override is not applied

Problem

The **MLflowConfig** resource is missing, has the wrong name, or exists in the wrong project. Alternatively, the associated secret does not exist.

Resolution

Ensure that an **MLflowConfig** resource exists and is named **mlflow**. Verify that the **mlflow-artifact-connection** secret is present in the namespace.

Kubernetes-namespaced authentication plugin cannot resolve credentials

Problem

The authentication plugin is missing a service account token (when running in-cluster) or an active **kubeconfig** context (when running locally).

Resolution

If running in-cluster, ensure the service account has a valid token. If running locally, verify that your **kubeconfig** context is active and points to the correct cluster and project by running **oc project**.

Artifact writes go to the default storage location

Problem

The **MLflowConfig** resource does not exist in the active project, or the **artifactRootSecret** is invalid.

Resolution

Create the **MLflowConfig** resource in your active project and verify that the **artifactRootSecret** contains the correct connection credentials.

JSON decode error: Expecting value

Problem

The Red Hat OpenShift AI authentication token is missing or invalid. Consequently, OpenShift AI receives an HTML response from MLflow instead of the expected JSON, causing OpenShift AI to prompt for a login.

Resolution

Log in to the OpenShift cluster by running the **oc login** command. Ensure that your environment uses the **MLFLOW_TRACKING_AUTH=kubernetes-namespaced** environment variable to authenticate requests.

3.5. MLFLOW VERSION COMPATIBILITY

The following information describes the compatible versions of MLflow and Red Hat OpenShift AI 3.4 GA.

Table 3.1. MLflow version compatibility and configuration

Item	Description
Deployed MLflow server version	3.10.1
Required MLflow SDK version	3.11 or later
Authentication plugin name	kubernetes-namespaced
Environment variable	MLFLOW_TRACKING_AUTH=kubernetes-namespaced

The following command installs the compatible version of the MLflow SDK:

```
+
| pip install "mlflow[kubernetes]>=3.11"
+
```



NOTE

MLflow SDK version 3.11 and later includes the **kubernetes-namespaced** authentication plugin by default.

3.6. MLFLOW STORAGE AND DATABASE COMPATIBILITY

The following table lists the MLflow storage and database configurations. The configuration settings vary according to your environment, for example production, development, or testing.

Table 3.2. Supported storage and database options

Storage area	Supported options
Artifact storage	S3 compatible object storage for production. File system for development and testing.
Database	PostgreSQL for production. SQLite for development and testing.
Artifact repository plugins	S3 and file.

3.7. TRACKING EXPERIMENTS WITH MLFLOW SDK

Use the MLflow software development kit (SDK) to log and track machine learning experiments. With the SDK, you can record parameters, metrics, and artifacts to a centralized tracking server for later analysis.

Prerequisites

- You have installed the MLflow SDK version 3.11 or later.
 - You have access to a Red Hat OpenShift AI data science project that has MLflow permissions configured.
 - You have configured a tracking URI and authentication. For more information, see [MLflow Kubernetes Authentication](#). Procedure
1. In your notebook, log experiments, parameters, metrics, and artifacts by using the MLflow SDK:
When the kubernetes-namespaced authentication plugin is configured, the tracking URI and workspace are resolved automatically.

```
import random
import time
import mlflow

mlflow.set_experiment("demo-experiment")

with mlflow.start_run(run_name="demo-run"):
    mlflow.log_param("model_type", "baseline")
    mlflow.log_param("feature_count", 3)
    for step in range(5):
        mlflow.log_metric("accuracy", 0.8 + random.random() * 0.2, step=step)
        mlflow.log_metric("loss", 0.5 - random.random() * 0.2, step=step)
        time.sleep(0.2)
```

2. Optional: To target a project other than the one associated with your credentials, set the workspace explicitly:

```
mlflow.set_workspace("<project-name>")
```

Troubleshooting

If you encounter errors when using the MLflow SDK to manage experiments or artifacts, see [Troubleshooting MLflow SDK](#).

3.8. CONFIGURE PROJECT-SPECIFIC S3 ARTIFACT STORAGE

By default, all projects use the artifact storage configured in the **MLflow** resource. You can override the artifact storage for a specific project by creating an **MLflowConfig** resource and a connection in that project. After you create these resources, MLflow resolves the artifact root from the override for any new experiments and runs that you create in that project. However, MLflow does not serve artifacts when you configure a per-project override. The client accesses the S3 bucket directly, so the client must have valid S3 credentials.

The **MLflowConfig** resource is namespace-scoped and must be named **mlflow**. It points to an S3 compatible object storage connection that holds the credentials and bucket information for the project.

Prerequisites

- You have installed MLflow.
- You have an S3-compatible object storage bucket with credentials.

Procedure

1. In your project, add a connection of type **S3 compatible object storage**. Set the resource name of the connection to **mlflow-artifact-connection**. Provide the following connection details:

- **Access key:** The S3 access key ID.
- **Secret key:** The S3 secret access key.
- **Endpoint:** The S3 compatible endpoint URL.
- **Region:** The S3 region. Optional if your storage provider does not require it.
- **Bucket:** The S3 bucket name.

If you are using the connections API instead of the dashboard, create a **Secret** with the annotation **opendatahub.io/connection-type-protocol: "s3"** and name it **mlflow-artifact-connection**. Setting an **AWS_DEFAULT_REGION** is optional. The required keys are:

- **AWS_ACCESS_KEY_ID**
- **AWS_SECRET_ACCESS_KEY**
- **AWS_S3_BUCKET**
- **AWS_S3_ENDPOINT**

2. Create an **MLflowConfig** resource named **mlflow** in the same project:

```
apiVersion: mlflow.kubeflow.org/v1
kind: MLflowConfig
metadata:
  name: mlflow
spec:
  artifactRootSecret: mlflow-artifact-connection
  artifactRootPath: mlflow-artifacts
```

artifactRootSecret must be **mlflow-artifact-connection**. The Custom Resource Definition (CRD) enforces this validation.



NOTE

artifactRootPath is an optional relative path that the system appends to the bucket root from the secret. For example, if the bucket is **ds-team-bucket** and **artifactRootPath** is **mlflow-artifacts**, the resolved artifact root becomes **s3://ds-team-bucket/mlflow-artifacts**. The path must be relative, must not use backslashes, and must not contain path traversal such as ...

